

SecPriv: A Unified Multi-Phase Agent Skill for LLM-Assisted Security and Privacy Code Review

Weicheng Wang, Nischay Gupta, Jie Pan, Piyush Nahar, and Gang Li

Meta, Menlo Park, CA, USA

{weichengw95,nischayg,jiepan,piyushnahar,morrisonli}@meta.com

Abstract. Code review must surface security weaknesses and privacy risks, yet automated tools treat the two as disjoint. We present SECPRIV, a unified agent skill that reviews both through one detector–validator methodology grounded in CWE and GDPR semantic memory. The unification claim covers the *source-to-sink*, *transformation-aware* subset the two surfaces share; on real CVEs, detection falls from 0.63 on the 41 in-subset cases to 0.47 on the remaining 23. On a code-audited 128-case benchmark over $N=5$ runs, removing the validator costs 0.17 precision and 0.24 true-negative rate at unchanged recall, so precision comes from the validator. A coverage-matched two-skill baseline ties on F_1 , which leaves one pass instead of two, at half the API cost, as the measured advantage. On 64 patch-labeled CVEs in 7 languages SECPRIV matches an unstructured LLM at file level (0.58 vs. 0.61) and halves its findings on already-fixed code (2.5 vs. 4.6 per file), but localizes only 0.31. Semgrep under one generic ruleset detects 0.02. We deploy Haiku as the default tier, tied with Sonnet on F_1 at a third of the cost, as a low-noise pass beside deterministic SAST. The contribution is reporting discipline, cost, and coverage rather than detection: detection tops out near 0.6 for every model we tried, the cross-surface claim rests on 8 cases, and privacy validation is still synthetic. All artifacts are released.

Keywords: agent skills · large language models · code review · security · privacy · GDPR · CWE · false-positive suppression

1 Introduction

Code review is a quality gate that organizations increasingly expect to catch *both* security weaknesses and privacy-risk signals before code reaches production [15, 14]. The two classes of defect arise together: a single change that adds an admin endpoint may open an authorization bypass and log a personal identifier at once. But the engineering practice that catches them is fragmented. Security review is supported by static application security testing (SAST) tools such as Semgrep and CodeQL [5, 6, 41], the Common Weakness Enumeration (CWE) taxonomy [3], and a growing literature on large-language-model (LLM) vulnerability detection [11, 12, 30, 39]. Privacy review is supported by separate tools [17, 7, 42], a separate reporting vocabulary (General Data Protection Regulation (GDPR) articles and personal-data classifications) [1], and a separate body of empirical work [18, 21]. Throughout, a *source* is an entry point for untrusted input or personal data, a *sink* is an operation where that data can cause harm,

and a *transformation state* records whether the data remains raw or has been hashed, encrypted, tokenized, or aggregated. Developers must otherwise install two skills, run two reviews, and reconcile the results, a cognitive load that surveys tie to security and privacy issues being deprioritized in practice [15, 18, 19].

The fragmentation persists despite the two domains sharing structural mechanisms. Both trace data from sources to sinks: in security, untrusted input to a dangerous sink (SQL query, shell, eval); in privacy, personal data to a logging, storage, analytics, or third-party sink [17]. Both then ask whether a transformation on that path defuses the risk: a SQL-injection candidate is suppressed if the input passes a parameterized API, and a privacy-leakage candidate may be downgraded if the identifier passes a salted hash, vault tokenization, or policy-enforced computation boundary [25, 26], though neither reduction establishes end-to-end protection. Framework knowledge serves both as well: an ORM that auto-parameterizes is the security analogue of a consent library that auto-gates processing. And both fail the same way in practice, when false positives erode developer trust until the tool is disabled [8, 9, 12]. This shared *source-to-sink, transformation-aware* core is the subset SECPRIV unifies, and we scope the claim to it (§3); logic- and runtime-dependent classes fall outside.

Recent agent skills have begun to address each surface in isolation. Preliminary experiments by the authors with separate security-only and privacy-only review skills showed that each surface can be addressed individually with high precision, but neither single-surface skill leverages the structural overlap between the two. SECPRIV is not a mere union: running the two surfaces as separate passes over the same input (a configuration we evaluate as a coverage-matched *two-skill* baseline, §8) matches the unified skill’s F_1 and recall, but costs twice as much, takes substantially longer, and cannot emit a cross-referenced cross-surface finding at all. Unlike our controlled-only predecessors, we test external validity on 64 real Common Vulnerabilities and Exposures (CVEs) across seven languages and multiple files.

We present **SECPRIV**, a unified multi-phase agent skill that performs security and privacy code review through a single methodology with shared infrastructure. The contributions are:

1. A *unified five-phase methodology* that adapts the detector–validator subagent pattern of AgenticSCR [13] to both surfaces, with a CWE-and-GDPR semantic memory and a shared transformation-aware filtering layer for code-level risk signals (§4).
2. A *controlled architecture study* on a 128-case benchmark (93 true-positive (TP) cases: 60 security + 30 privacy + 3 cross-domain; 35 true-negative (TN) cases: 15 security + 15 privacy + 5 cross-domain; 30 MITRE/OWASP/GDPR-sourced categories): a detector-only ablation (paired $F_1 - 0.109$, TN-rate -0.24 , significant over 5 runs) shows the validator carries precision; a *matched-schema* two-skill baseline isolates architecture from coverage (the F_1 difference is within noise, and the pooled TN advantage does not survive the held-out split, leaving 2× lower cost and lower latency at matched recall as the measured advantage); and a 12-case micro-benchmark isolates the shared transformation classifier’s mechanism on adequately-transformed flows that generic corpora under-sample, a mechanism demonstration only, since the classifier has no measurable effect on either realistic corpus (§7.2, §8).

3. A *real-world external-validity probe* of 64 CVEs across 7 languages, built from public GitHub Security Advisories with *patch-derived* labels (pre-patch file = true positive, post-patch file = fixed-version control for that CVE), with a matched baseline comparison: SECPRIV matches an unstructured LLM’s file-level detection at roughly half its fixed-code finding rate, while Semgrep under one generic ruleset has much lower coverage on this CVE set (§7.3).
4. An *end-to-end evaluation harness*: SKILL.md, all three evaluations (controlled benchmark, real-CVE probe, transformation micro-benchmark) and their ground truth, a real-CVE ingestion pipeline, a tiered patch-diff scorer, JSON parser with retry logic, alias normalizer, and per-configuration logs.

2 Background and Related Work

Modern code review for security and privacy Code review catches some security issues but leaves systematic gaps: Charoenwet et al. [14] found concerns across 35 of 40 CWE-699 categories in OpenSSL/PHP reviews, many still unfixed, and Braz and Bacchelli [15] found that developers rarely mention security spontaneously. Privacy has a different bottleneck: Theophilou and Kapitsaki [18] report that developers most want automated privacy-risk detection. Braz et al. [16] show focused reviewer instruction improves detection, motivating skill-style guidance over long checklists.

Static and LLM-based vulnerability detection Traditional SAST encodes pattern rules over data flow [4–6], precise on covered patterns but weak on application context [9]. LLM-based review complements this: Beljulji and Gür [12] identify prompt engineering with curated context as the dominant approach, while work on generated-code security and transformer detection shows promise and residual risk [29, 31]. AgenticSCR [13] is closest architecturally, pairing a detector with a validator that filters against CWE memory; SECPRIV adopts that decomposition and extends it across two surfaces, with GDPR articles supplementing CWE.

Privacy code analysis Tang et al. [17] provide the most directly relevant prior work, formalizing personal-data processing as a flow between 9 source types and 5 sink types annotated with a sensitivity transition, with their Semgrep-based tool reaching ~90% precision on processing-flow detection. SECPRIV adopts this taxonomy as the privacy half of its source/sink memory and extends it with a transformation-state classifier (raw, salted-hash, aggregated, encrypted, tokenized). The analysis is grounded in Privacy by Design [33] and Solove’s taxonomy of privacy harms [34]; code-level analysis of contact-tracing protocols illustrates why policy claims must be tied to concrete flows rather than to documentation [24].

LLM evaluation realism SecureVibeBench [20] and SusVibes [36] show that agentic coding systems remain far from uniformly secure on repository-level tasks, and scaffold design, prompt context, and stochastic outputs all affect measured results [12, 10, 37]. This motivates our repeated-run reporting, cost accounting, and comparison between a structured skill and unstructured prompts.

Skills gap and positioning Surveys report that 75% of engineering leaders consider application security highly important while only 23% believe their teams have adequate security skills [19]; Theophilou and Kapitsaki [18] report a complementary privacy gap of tooling scarcity rather than under-attention, with developers requesting “modular, reusable components for common privacy tasks.” These two gaps motivate one skill an organization installs once and invokes uniformly, rather than two competing for reviewer attention. Security scope, privacy scope, agent packaging, and held-out negative controls are easy to conflate. Semgrep/CodeQL [5,41] and AgenticSCR [13] cover security only; FlowDroid/Privado [7,42] and Tang et al. [17] privacy only; none report held-out negative controls. SECPRIV’s contribution is the combination: one agent workflow over both surfaces, evaluated on 128 controlled cases with a held-out TN subset plus a 64-CVE real-world probe.

3 Scope, Threat Model, and Reviewer Setting

We assume a standard pre-commit/pull-request review setting [13]. A developer commits code that may contain security weaknesses, privacy-risk signals, or both. The agent receives the changed file (or, in the repository-extension setting, the diff plus a small number of related files) and has read-only access to the repository for context. Its output is a review finding, not a proof of exploitability or of adequate end-to-end data protection.

Security adversary An external attacker who can submit untrusted input to any externally reachable endpoint and may control values previously stored in semi-trusted locations (databases, caches, config tables, feature flags). Collection points include HTTP parameters, headers, bodies, webhook payloads, uploads, URL components, queue events, and values read back from attacker-influenceable storage. Following established practice, application-level configuration is explicitly *not* trusted, in contrast to OS environment variables. The skill must construct a concrete exploitation path from such a source to a dangerous sink before reporting a finding.

Privacy adversary A privacy reviewer examining how personal data is handled, plus the user whose data is at stake. We use GDPR [1] and CCPA [2] as reporting vocabulary for their widely used article-level hooks, but the skill does not assess an organization’s policies, contracts, or governance processes. Privacy risks in code are documented across mobile telephony [22], IoT [23,40], and contact tracing [24], motivating review across the stack. Personal data flowing to a sink is flagged on five conditions visible in code: inadequate transformation, processing with no explicit basis in the code path, retention beyond purpose, re-identification risk from combined quasi-identifiers, and reuse beyond the original collection intent (*purpose creep*). Each finding maps to a GDPR article as explanatory metadata only, naming the principle the code-level signal relates to.

Collection-point assumptions The detector traces only collection points visible in source code and repository context; it does not prove that an upstream service, SDK, or identity provider is uncompromised, and treats such outputs according to the trust boundary

visible at the reviewed code. As in practical code review, compromise of external collection infrastructure is out of scope unless the change itself exposes that boundary.

Cross-surface concerns Several real defects span both surfaces: an authorization bypass that exposes personal data is simultaneously a security and privacy issue; a logging statement that leaks both a session token and an email is the same code line under both rubrics. A unified skill should handle these as one finding (or two findings explicitly cross-referenced) rather than two unrelated entries on two separate dashboards.

Scope of the unification claim We claim unification only for the *source-to-sink, transformation-aware* subset both surfaces share: *did tainted or personal data reach a sink in an inadequate transformation state?* This covers the injection-family security categories (SQLi, command, path, XSS, SSRF, SSTI, XXE, eval, deserialization, open-redirect) and the leakage/transformation privacy categories (pii-leakage, insecure-storage, re-identification, third-party-sharing, cross-border-transfer, data-minimization); one transformation-state classifier governs suppression on both (§4.4). We do *not* claim to unify classes that need cross-procedure or runtime reasoning: authorization-logic and insecure-direct-object-reference (IDOR) flaws, configuration errors, state-machine bugs, non-pattern crypto misuse, race conditions/TOCTOU, and consent-timing/purpose logic. For these SecPriv is *complementary* to logic-focused review and SAST. The real-CVE probe confirms this boundary empirically (§7.3).

4 The SECPRIV Framework

SECPRIV is a single agent skill that performs both security and privacy-risk review through a five-phase pipeline. What distinguishes it from a naive concatenation of the two prior skills is a *detector–validator decomposition* adapted from AgenticSCR [13], working over a *shared semantic memory* that grounds judgments on both surfaces in authoritative references (CWE for security, GDPR articles for privacy-risk signals).

The pipeline runs Context → Detect → Validate → Map → Findings within a single model call, with shared semantic memory supplying references, source/sink categories, framework facts, and transformation states to the detector and validator; the output is one cross-surface finding list.

4.1 Architectural Components

The *detector* (a_d) executes Phases 1–3 against the changed code, receiving the methodology, semantic memory, and read-only repository context, and emitting an over-approximate finding set: any plausible match to one of the thirty canonical categories (§4.4), with confidence, surface, category, line, and evidence. The *validator* (a_v) executes Phase 4 over that output, applying the shared suppression layer (§4.5) and a 0.8 confidence threshold to revise, suppress, or pass each finding. The *mapper* (a_m) runs Phase 5, attaching a CWE-ID or GDPR article and fixed-table remediation language to avoid stochasticity in the final report. All three are roles within one model call.

Semantic memory (M_s) is a ~ 4.7 k-token static document supplying CWE-699 descriptions, GDPR Articles 5–7/28/44–49 summaries, Tang et al.’s source/sink taxonomy [17], transformation states, and ~ 90 framework-awareness facts. Each JSON finding carries surface, canonical category, severity, confidence, file/line, description, exploitation path or privacy rationale, and remediation.

4.2 Phase 1: Repository Context Research

Before assessing any change, the detector establishes context: **framework detection** identifies ORMs, sanitizers, auth middleware, template engines, and consent libraries from imports and manifests; **trust-boundary mapping** classifies visible interfaces as untrusted (HTTP fields, webhooks, uploads), semi-trusted (database values, config tables, feature flags), or trusted (OS environment variables, constants); and **personal-data inventory** identifies PII, financial, health, or location data and visible consent mechanisms. The output is a ≤ 300 -token context summary prefixing Phases 2–3.

4.3 Phase 2: Source/Sink and Transformation Flow Mapping

The detector traces data from collection points (API endpoints, form handlers, webhooks, SDK callbacks) through processing to terminal sinks (databases, caches, logs, third-party APIs, shells, eval/exec, template renderers). This is review-grade local flow reconstruction, not whole-program taint proof: indirect calls, dynamic dispatch, generated code, and upstream services are included only when visible in context. For each flow, it records the source category, sink category [17], and *transformation state* at the sink.

The transformation classifier is the mechanism that lets one suppression layer serve both surfaces: Table 1 shows how each state governs findings on both surfaces through a single check, rather than requiring each surface to re-verify the same transformation. Its empirical reach is narrow: the classifier can only act on flows that *are* adequately transformed, and such flows are rare in both of our evaluation corpora. Its effect is consequently statistically invisible on the 128-case benchmark and the real-CVE probe alike, and is measurable only on a purpose-built micro-benchmark (§7.2). The mechanism works, then, but we cannot yet show it changing outcomes on input we did not construct.

For example, when an email is salted-hashed before reaching a logging sink, the *raw* state becomes *salted-hash*. This can suppress both a *pii-leakage* candidate (a lower-risk pseudonymized form, though still potentially personal data under GDPR Art. 4(5)) and an *xss* stored-variant candidate (the hash is opaque). Without the shared classifier, each single-surface skill must re-verify the same transformation and may reach inconsistent judgments.

4.4 Phase 3: Dual-Surface Vulnerability Assessment

The skill defines 30 canonical categories (20 security, 10 privacy), all evaluated by the single-file benchmark in §5. Second-order vulnerabilities (*second-order-sqli*, stored-variant *xss*) are exercised by combining write and read paths in one file; their accuracy is reported in §7.4.

State	Adequate?	Sec. suppresses	Priv. suppresses
raw	No	(none)	(none)
salted-hash	Yes	xss (stored variant)	pii-leakage
unsalted-hash	No	(none)	(none; re-id risk)
k-aggregated	Yes	—	pii-leakage, 3p-sharing
encrypted (KMS)	Yes	hardcoded-secret	insecure-storage
tokenized (vault)	Yes	—	pii-leakage, 3p-sharing
base64 / hex	No	(none)	(none; reversible)

Table 1. Transformation-state classifier. “Adequate” = the validator’s R5 rule suppresses the listed privacy categories; “Sec. suppresses” / “Priv. suppresses” = which categories are suppressed on each surface. A single transformation check simultaneously governs both surfaces. Rows cover all 30 implemented categories, not only the unification subset of §3. Stored-variant *xss* is an alias normalized to *xss* (`runner/aliases.json`, 181 entries).

Security categories (20 total; CWE-mapped). *sql-injection*, *command-injection*, *path-traversal*, *xss/xss-dom*, *auth-bypass*, *hardcoded-secret*, *deserialization*, *eval-injection*, *infinite-loop*, *agent-csrf* (CWE-352 generalization for LLM agents), *second-order-sqli*, *ssrf*, *open-redirect*, *crypto-misuse*, *xxe*, *ssti*, *csrf*, *race-condition*, *insecure-logging*. Each category maps to a CWE entry (full mapping in the released artifact). For second-order vulnerabilities, the detector explicitly searches for write paths reachable by external users; application-level configuration is treated as attacker-influenceable.

Privacy categories (10 total; GDPR-referenced). Cited GDPR articles name the privacy principle that motivates each category; the skill produces code-level risk signals only. *pii-leakage* (Art. 5(1)(f)), *data-retention* (Art. 5(1)(e)), *consent-bypass* (Art. 6, 7), *third-party-sharing* (Art. 28, 44–49), *re-identification-risk* (Art. 4(5); zip + DOB + gender uniquely identifies 87% of the US population [32]), *data-minimization* (Art. 5(1)(c)), *purpose-creep* (Art. 5(1)(b)), *insecure-storage* (Art. 32), *cross-border-transfer* (Art. 44–49), *right-to-erasure* (Art. 17 / CCPA §1798.105).

The detector emits one finding per (line, category) pair even when multiple flows converge on a line, so a logging statement leaking both a session token and a personal identifier yields two findings cross-referenced by a shared `flow_id`.

4.5 Phase 4: Validator and Filtering

The validator subagent applies seven suppression rules independently to each detector finding.

R1. Exploitation path verification. For security findings, the validator must construct a concrete end-to-end exploitation path from untrusted input to the dangerous sink with no sanitization on the path. For privacy findings, the validator must verify that the transformation state at the sink is inadequate (raw, unsalted-hash, or reversibly encoded as base64/hex) for the data classification.

R2. Framework auto-protection. Suppress findings on paths protected by an established framework (ORM-parameterized queries, React JSX, Jinja2 escaping, declared consent libraries). **R3. Test/documentation scope.** Suppress credential and PII findings in test files, documentation, and synthetic fixtures, but *not* logic findings (an auth

bypass in a test endpoint may indicate a real flaw). **R4. Trusted source.** Values from OS environment variables are not hardcoded secrets. **R6. Same-purpose reuse.** Personal data reused for its collection purpose (registration email for password reset) is not purpose creep. **R7. Unsuppressible evidence.** High-certainty patterns (SECRET_KEY, PRIVATE_KEY, aws_secret_access_key, plain-text national IDs and passwords) are never suppressed, following defensive SAST design [12]; R7 matches literal values only, so an environment-variable read of the same name stays suppressed under R4.

R5. Transformation adequacy (privacy-specific). If the transformation state at the sink is adequate per Table 1 (salted-hash, k -aggregated, encrypted with a key management service (KMS), or vault-tokenized), the privacy finding in the categories listed for that state is suppressed at the code level; unsalted hash is *not* adequate. R5 assumes deployment context the skill cannot see: managed key rotation, an upstream aggregation threshold, a token-vault boundary the code does not bypass. A reviewer should confirm these before acting on a suppression, and the validator can still misclassify an adequately protected flow as *insecure-storage*.

Confidence threshold and output. Findings below 0.8 confidence are downgraded to an advisory “manual review” channel; suppressed findings become INFO rather than disappearing, preserving an audit trail. The validator normalizes the detector’s informative sub-category names (e.g. `sql_injection_fstring`) to the 30 canonical labels via a fixed alias table. Phase 5 then attaches the CWE-ID or GDPR article and a fixed-table remediation indexed by canonical category, grounding the report in an authoritative reference [13]. The pipeline returns one JSON array; cross-surface issues on a line become two findings sharing a `flow_id`, so a review queue can present them together or deduplicate. The developer receives one reference-grounded list, which is what the unification produces.

5 Evaluation Datasets

We evaluate on two datasets of equal standing: a *controlled* synthetic benchmark for clean per-category diagnostics and ablations (§5.1), and a *real-CVE probe* for external validity on genuine, multi-language, sometimes multi-file vulnerabilities (§5.2). Neither subsumes the other. The controlled set isolates mechanisms that real code buries under confounds; the probe tests whether those mechanisms survive contact with code we did not write.

5.1 Controlled 128-case benchmark

This benchmark is a controlled synthetic diagnostic, not a real-world corpus: every case is a purpose-built self-contained Python file. This enables clean per-category and held-out-TN measurement but excludes repository-scale flow, multi-file privacy paths, framework-version variation, and the long tail of real bugs. We construct 128 cases covering 30 canonical categories (20 security, 10 privacy) with 3 construct variants per category (90 cases), 30 near-miss true negatives (10 held out from suppression-rule calibration), and 8 cross-domain cases (3 TP + 5 TN), giving 93 TP cases and 35 TN cases in total. A supplementary `category_sources.md` cites the public source

(MITRE Top 25 2025, OWASP Top 10 2021, or GDPR Enforcement Tracker) for each category.

Construction principles are: realistic imports in 30–60 lines-of-code (LOC) files; true negatives targeting specific discrimination boundaries instead of trivially safe code; structurally distinct variants per category; and category $\times$ line-proximity scoring (± 10 lines).

Security true positives (60 cases) The 20 categories cover 8 MITRE Top 25 entries applicable to Python [28], 11 further OWASP Top 10 or common CWE categories [27], and one agentic category (*agent-csrf*), with variants exercising different API forms (f-string, `.format()`, raw SQLAlchemy `text()` for SQL injection); memory-safety CWEs are excluded by language scope.

Privacy true positives (30 cases) The 10 categories cover GDPR Enforcement Tracker top-cited articles applicable to code, including *insecure-storage* (Art. 32), *cross-border-transfer* (Art. 44–49), and *right-to-erasure* (Art. 17 / CCPA §1798.105).

Near-miss true negatives (30 cases) 15 security + 15 privacy, each paired with a TP construct variant by single-edit transformation. Of these, 20 are *calibration* cases (visible during suppression-rule development) and 10 are *held-out* (flagged with `holdout: true` in the ground truth before any rule editing; contents not inspected until final TN-rate measurement). The held-out/calibration split addresses the concern that calibrating rules on the same TN cases used for evaluation constitutes train-on-test; the two subsets are scored separately in §7.4, since the pooled TN column of Table 2 cannot answer that question on its own.

Cross-domain cases (8 cases) Three TPs and five TNs exercise the unification claim: unauthenticated PII forwarding, unsigned webhook logging, and second-order SQLi over stored PII, plus protective combinations such as JWT auth with salted hashing, HMAC verification with hashed summaries, and encrypted-at-rest processor flows.

Multi-label ground truth Single-label scoring under-credits valid secondary findings: an unauthenticated command-injection endpoint is also an *auth-bypass*, and a hard-coded AES key may be both *hardcoded-secret* and *crypto-misuse*. We therefore re-annotated the benchmark through an independent code audit blind to model outputs, which corrected 3 mislabels and added 29 net code-verified secondary labels across 26 cases (32 label touches including the relabels, 96→125 expected findings); a companion audit confirmed all 35 TN cases are clean. All results in §7 use this multi-label ground truth.

5.2 Real-CVE probe

To measure external validity we build a second dataset from *real* vulnerabilities with *patch-derived* labels rather than author hand-labeling, drawn from the public GitHub Advisory Database (`/advisories?type=reviewed`) across six ecosystems (pip, npm, Maven, Go, Composer, RubyGems; npm spans JavaScript and TypeScript, hence 7

languages). An automated pipeline (released with the artifact) keeps a reviewed advisory only if its CWE maps to a canonical security category, it links a public fix commit modifying a non-generated source file in a supported language, and the changed region can be anchored back to the pre-patch file; we discard documentation/test-only changes, vendored or minified code, and oversized files. For retained advisories it fetches every changed in-scope file at the *parent* SHA (vulnerable) and *fix* SHA (fixed), recording CVE/GHSA, repository, and SHA for reproduction. The pre-patch file is a positive for the patched family; the post-patch file is a fixed-version control for that CVE, not a proof the file is vulnerability-free. The pilot yields 64 **CVEs / 128 paired files across 7 languages** (Python, JavaScript, TypeScript, Java, Go, Ruby, PHP), 29 multi-file, spanning 15 categories. The probe is *security-only*: privacy defects rarely receive CVEs, so no comparable real-world privacy corpus exists, which remains an open problem.

6 Experimental Setup

Each dataset is evaluated on its own track, with a third track isolating the transformation classifier. All runs use temperature 0 with up to 3 retries on empty responses.

Controlled-benchmark track Over $N=5$ runs each we compare four Sonnet (Anthropic Claude) configurations: SECPRIV (full unified skill; primary), a *detector-only* ablation (Phases 4–5 disabled), a coverage-matched *two-skill* baseline, and a *no-skill* baseline (minimal “review this file” prompt with the output schema). We add SECPRIV-HAIKU (same model family) under the same protocol for cost comparison. *Scoring*: precision, recall, F_1 , and true-negative rate on the 35 TN cases (20 calibration + 10 held-out + 5 cross-domain), matched by canonical category and line-proximity (± 10) against the multi-label ground truth.

Construction of the two-skill baseline This is the paper’s main architectural comparison, so its construction matters. Each pass receives the full methodology and 30-category vocabulary as *context*, but emits findings for one surface only, staying silent on the other even on the same line. The passes are therefore *disjoint in what they emit*, 20 security categories for one and 10 privacy for the other, so the union cannot manufacture duplicate rows; we measure 0 exact (category, line) collisions per run across all five runs. Sharing the vocabulary as context is what makes the comparison coverage-matched, so residual differences reflect one pass versus two rather than one configuration knowing more categories. It does *not* model two independently authored skills with different methodologies and vocabularies, which would differ along several axes at once, the confound we remove. The comparison is correspondingly narrow: it says nothing about how SECPRIV fares against two off-the-shelf skills.

Real-CVE probe track On the probe (§5.2) we compare six systems: SECPRIV and an unstructured same-model LLM (no-skill, identical schema) on both Sonnet and Haiku; the unstructured LLM plus a one-paragraph conservatism instruction (Sonnet); and **Semgrep** [5] (OSS `p/security-audit+p/owasp-top-ten`, 690 rules) as the SAST baseline. *Scoring*: whole-file “clean=TN” and exact category×line matching are meaningless on real code, so we score the *paired diff*: file-level detection (a family-matching

finding anywhere in the vulnerable file), localized detection (± 10 lines of the patched region, insertion-only fixes anchored to the pre-patch bracket), and, as a false-alarm proxy, findings emitted on the *fixed* file. The fixed-file count is not a conventional false-positive rate, since the post-patch file may contain unrelated weaknesses; it estimates how much noise a reviewer still sees after the known fix. A category canonicalizer is applied uniformly to every system, so a baseline’s free-form labels (“Path Traversal”) are credited exactly like canonical ones; without it the unstructured LLM is unfairly penalized.

Transformation-classifier micro-benchmark The classifier-ablated configuration is statistically indistinguishable from full SECPRIV on *both* datasets above, because both are dominated by `raw`→`sink` flows that never exercise the classifier. We therefore add a 12-case micro-benchmark that concentrates its trigger cases: personal data reaching a sink but transformed by salted-hash, KMS-encryption, $k \geq 5$ aggregation, vault tokenization, or `bcrypt` (adequate⇒TN), against `base64/unsalted-hash/hex/raw` (inadequate⇒TP controls), plus a cross-surface pair. We compare the full skill against the classifier-and-R5-disabled variant, both on Sonnet (§7.2).

7 Results

Aggregate 128-case metrics (both Sonnet and Haiku), the real-CVE detection/noise, and the classifier micro-benchmark are reported as $N=5$ mean with 95% CIs; the held-out-TN and cross-surface analyses also report $N=5$ means, while the per-category and failure-mode breakdowns are single-run and should be read as point estimates.

7.1 Aggregate Performance

Configuration	P	R	F1	TN	Lat.	\$/case
SecPriv-Sonnet	.75±.04	.92±.03	.83±.04	30	74	.024
SecPriv-Haiku	.83±.02	.80±.02	.82±.02	32	46	.008
Two-skill baseline	.70±.03	.89±.02	.78±.02	26	118	.048
Detector-only	.58±.03	.95±.02	.72±.02	21	76	.024
No-skill	.05±.01	.15±.04	.08±.02	9	31	.012

Table 2. Aggregate performance on the 128-case benchmark (93 TP cases + 35 TN cases: 60+30+3 TP; 15+15+5 TN), all Claude Sonnet except the Haiku row. Values are **mean ± 95% CI over $N=5$ runs** (small-sample t), scored against the **code-audited multi-label ground truth** (125 expected findings). TN = mean correctly-cleared cases (rounded) out of 35 near-miss true negatives; Lat. = mean wall-clock seconds/case; \$/case = per-case API cost modeled from token counts at list prices (Sonnet \$3/\$15, Haiku \$1/\$5 per M in/out tokens), so the $2\times$ two-skill cost follows by construction from two model calls vs. one. The *two-skill baseline* gives both single-surface passes the identical 30-category schema (coverage-matched). Single-label results (which under-credit precision) are in the released artifact.

The five configurations separate along precision far more than along recall, and the paragraphs below take each in turn.

SecPriv and the two-skill baseline Recall for the unified skill is near-ceiling at 0.92, and its lower precision of 0.75 comes from co-findings the validator emits on TP cases, many of them real secondary vulnerabilities the conservative answer key does not credit (§7.4). The coverage-matched two-skill baseline falls within noise of it on F_1 , 0.78 against 0.83 (§8).

Haiku versus Sonnet Under multi-label ground truth the two model tiers are statistically tied on F_1 at 0.83 and 0.82, with a paired difference of 0.015 ± 0.047 whose direction we cannot resolve. Haiku is the more precise (paired +0.07) and Sonnet the more complete (+0.12 recall); manual inspection attributes Sonnet’s lower precision to the same uncredited secondary findings.

No-skill baseline The bare prompt reaches $F_1 = 0.08 \pm 0.02$ at 0.05 precision and 0.15 recall, and the paired gap to SecPriv (+0.755, 95% CI [0.71, 0.80]) is numerically our largest effect. It is also the one most open to misreading. The bare prompt is not quiet: it emits 371 findings per run, *more* than SecPriv’s 153. But 330 of those (89%) carry a category label outside the 30-term canonical vocabulary even after the alias table is applied, against 0/153 for SecPriv, and an unmatched label cannot score as a true positive under category×line matching. The 0.08 therefore measures schema conformance much more than it measures the ability to notice a vulnerability. Detection is compared on equal terms only on the real-CVE probe (§7.3), where one canonicalizer is applied uniformly to every system and the same unstructured Sonnet prompt reaches 0.61 against SecPriv’s 0.58. On the controlled benchmark, speaking the answer key’s vocabulary is most of what the skill contributes.

Validator ablation The detector-only configuration disables Phases 4 and 5. Across 5 runs SecPriv beats it by $F_1 +0.109$ (95% CI [0.09, 0.13]), precision +0.174, and TN-rate +0.240 ([0.18, 0.30]), all significant, while recall is unchanged. The detector surfaces candidates and the validator suppresses false positives, supporting the AgenticSCR-inspired [13] decomposition.

7.2 Transformation-Classifier Micro-benchmark

Table 3 reports $N=5$ means on the 12-case mechanism benchmark of §6, which concentrates the adequately transformed flows the classifier can act on and that are rare in both realistic corpora.

Config	TN-rate (adequate)	Prec.	Recall
Full skill	0.86	0.50	0.77
Classifier ablated	0.60	0.39	0.73

Table 3. Transformation-classifier micro-benchmark (12 cases, Sonnet, $N=5$ mean). Ablating the classifier significantly lowers precision (paired +0.11, 95% CI [0.01, 0.21]) and the true-negative rate on adequate-transformation cases (paired +0.26, [0.11, 0.41]), while recall on the inadequate/raw controls is unchanged (+0.03, n.s.)—so the classifier discriminates rather than blanket-suppresses.

Ablating the classifier turns adequate-transformation cases into false positives, whereas both configurations still catch inadequate controls (base64, unsalted-hash, hex, raw) at similar recall. The classifier therefore discriminates rather than blanket-suppresses; on 12 cases the precision and TN-rate gains are significant while F_1 is not.

Bounds on this result The absolute numbers are weak: the full skill reaches only 0.50 precision, roughly one false positive per true one. Every case here sits on the raw/adequate decision boundary by construction, so 0.50 describes the mechanism under maximal stress; the deployment-relevant figure is 0.75 on the 128-case benchmark. The ablation delta on its own would suggest the classifier resolves these cases more cleanly than it does. The comparison is also internal, and supports only the claim that the classifier helps where adequately transformed flows occur. Since those flows are rare outside this benchmark, its benefit is established on constructed inputs alone and carries no weight in the paper’s evidence until a corpus shows them to be common in real review traffic.

7.3 External Validity: Real-CVE Probe

Table 4 reports the real-CVE probe (64 CVEs, 7 languages, patch-derived labels), the paper’s primary external-validity evidence.

System	Detect [95% CI]	Local.	Noise _{fixed}
Semgrep (SAST) [§]	.02 [.00, .08]	.00	0.1
No-skill LLM (Haiku) [§]	.53 [.41, .65]	.28	4.1
No-skill LLM (Sonnet)	.61±.04	.28	4.6
No-skill conservative (Sonnet)	.48±.05	.22	1.5
SecPriv (Haiku)[§]	.56 [.44, .68]	.30	2.5
SecPriv (Sonnet)	.58±.04	.31	2.5

Table 4. Real-CVE probe (64 CVEs). Detect = file-level detection of the CVE’s vulnerability family; Local. = detection within ± 10 lines of the patched region, the stricter signal for actionable review; Noise_{fixed} = mean findings per *already-fixed* file (residual-noise proxy, not a conventional false-positive rate). Sonnet SecPriv / no-skill / conservative are $N=5$ mean $\pm$ run 95% CI; [§] single-run (Wilson 95% CI in brackets; Semgrep is deterministic). *No-skill conservative* is the unstructured LLM plus a one-paragraph “be careful” instruction. A hand-adjudication of a 25-CVE sample against the patched code agreed with the automatic verdict on 25/25 cases, validating the metric.

Coverage and localization Every LLM configuration detects 0.53–0.61 of the real CVEs, against 0.02 (1/64, at the wrong line) for Semgrep’s OSS `p/security-audit+p/owasp-top-ten` configuration. One generic ruleset produces a large gap, but it does not establish a universal SAST result: advisory-specific rules, commercial configurations, or CodeQL may cover different subsets. File-level detection measures whether a reviewer recognizes the CVE family, but actionable review also requires pointing near the faulty code, and SECPRIV-Sonnet localizes only 0.31 of CVEs within ± 10 patched lines against 0.58 at file level. The probe is therefore evidence of useful external validity, not of repository-scale repair readiness.

Detection versus residual noise Over 5 runs the unstructured LLM is a strong but noisy detector: on Sonnet it detects 0.61 against SECPRIV’s 0.58, a gap that does not clear significance (paired -0.03 , CI $[-0.07, 0.01]$). The two diverge instead on fixed-version controls, where the unstructured prompt emits 4.6 findings per file against 2.5, a paired difference of -2.1 (CI $[-2.7, -1.5]$). At the patched region (± 10 lines, `runner/score_cve_probe.py`), it emits about twice as many off-target findings (25 vs. 11 on run R1, representative of $N=5$). SECPRIV’s advantage is lower residual noise, not detection superiority.

A conservatism-instructed baseline The *no-skill conservative* prompt cuts noise to 1.5 findings/fixed file, below SECPRIV, but detection falls to 0.48; the 0.10 shortfall against SECPRIV clears significance (CI $[0.04, 0.16]$). Conservatism slides down the precision–recall curve, while the detector–validator structure finds a better balance on it.

The detection ceiling No system exceeds 0.61 file-level detection (0.31 localized) on this probe, and ~ 0.6 is typical. Misses are dominated by missing-guard/broken-access-control fixes, which are not source-to-sink flows. This matches our scope claim (§3): SecPriv-Sonnet detects injection-family source-to-sink CVEs at 0.63 ($n=41$) versus 0.47 on the remaining 23, of which authorization/missing-guard flaws are 16. We split by injection family because the probe is security-only, so the 4 CVEs carrying privacy-category labels fall in the remainder even though §3 scopes those categories in.

7.4 Further Analyses on the Controlled Benchmark

These diagnostics use the controlled 128-case benchmark and, unless noted, a single run.

Per-category recall SecPriv-Sonnet detects 26/30 categories at 100% recall. The four below 1.00—hardcoded-secret (6/7), pii-leakage (6/7), third-party-sharing (4/5), crypto-misuse (3/4)—are secondary labels, harder than planted primaries. The single-label “hard” categories (*xss*, *second-order-sqli*) both rise to 1.00 after label correction.

Held-out versus calibration true negatives Table 2 pools all 35 TN cases, which hides the very comparison the held-out split was designed to enable, so we separate it here ($N=5$ means). SecPriv-Sonnet clears 17.8/20 calibration TNs (0.89 ± 0.05), 7.8/10 held-out TNs (0.78 ± 0.10), and 4.2/5 cross-domain TNs. The held-out minus calibration gap is -0.11 ± 0.12 : lower on unseen cases, as expected, but not distinguishable from zero at $N=5$. We read this as generalizable discrimination rather than memorized exceptions, noting that 10 cases is a weak test that would miss modest overfitting. The validator’s advantage is likewise not a calibration artifact: on held-out cases alone SecPriv beats detector-only by $+0.18 \pm 0.16$ TN-rate (7.8 vs. 6.0 of 10), so the conclusion of §7.1 survives on untuned data. Against the two-skill baseline, however, the held-out margin is $+0.10 \pm 0.15$ (n.s., 6.8/10), so the pooled TN advantage rests mainly on calibration cases and should be discounted accordingly.

Detection–suppression stability Across 5 runs, 26/35 TN cases are stable (25 correct, 1 wrong); the 9 intermittent cases are protective-construct recognitions such as decorators, consent gates, and encryption boundaries.

Cross-surface findings These cases are the only direct test of the unification claim, so we report all 5 runs rather than a point estimate. SecPriv-Sonnet reaches F_1 0.79 ± 0.09 (precision 0.65 ± 0.13 , recall 1.00, 4.2/5 TNs cleared) versus 0.02 ± 0.04 for the no-skill baseline. The result is stable across runs but rests on 8 cases, 3 TPs and 5 TNs, which makes it the thinnest evidence in the paper: a ± 0.09 interval over three positives reflects run-to-run agreement, not sampling error over a population of cross-domain defects. Against the two-skill baseline the difference is $+0.05 \pm 0.12$ (n.s.), so unification shows no measurable quality advantage even here. What it does provide, and no surface-restricted pair can, is the artifact itself: paired security+privacy findings sharing a `flow_id` in one queue. At this sample size we can argue the design property but not demonstrate it, and a larger cross-domain corpus is the most valuable next experiment.

Failure modes Manual inspection shows that many residual false positives are real co-findings on TP cases that the answer key does not credit; genuine false positives are far fewer (e.g. 6 on TN cases in run R1 per `runner/analyze_failures.py`). Single-label scoring additionally labels 52 TP-case extras spurious, most of which multi-label credits. Reported precision is therefore a conservative lower bound.

Robustness checks Cached-emission diagnostics show the scoring is not doing the work. Raising the confidence cutoff from 0.8 to 0.9 leaves F_1 flat-to-rising ($0.83 \rightarrow 0.85$) before collapsing at 1.0. SecPriv emits canonical labels directly—no finding required an alias in any of the five runs—so the alias table (181 entries) benefits only the no-skill baseline (§7.1). Hardcoded-secret is the only category a regex could catch, and only for the 6 of its 7 labels that are literal patterns; every other category requires semantic flow reasoning.

8 Discussion

The ablations and the real-CVE probe point the same way: SECPRIV’s value lies in how findings are filtered and reported rather than in finding more of them. The rest of this section works through what that implies for the architecture, for what unification is worth against running two separate skills, for cost, and for the limits of a skill that lives entirely in a prompt.

Where the accuracy comes from The detector-only ablation separates two effects that the aggregate numbers combine. Methodology supplies most of the gain from a bare prompt to the detector, while the validator supplies the precision and TN-rate that make the output deployable, echoing AgenticSCR [13]. The F_1 jump from no-skill to full skill is our largest effect, but §7.1 attributes most of it to output-schema conformance, and on real CVEs the unstructured prompt detects about as well as the skill. What a skill buys, then, is a stable machine-consumable vocabulary and a suppression layer rather than new detection capability. For skill authors this reorders the investment: schema discipline and suppression rules pay off first, and the underlying model sets the detection ceiling.

Unified skill versus two skills A two-skill baseline must share the unified skill’s 30-category vocabulary as context while keeping the two passes disjoint in what they emit, else the comparison conflates coverage with architecture; §6 gives the construction and confirms the union introduces no duplicate findings. With coverage matched, the two configurations are indistinguishable on F_1 , the paired difference being $+0.048 \pm 0.050$. Pooled over all 35 TN cases the unified skill suppresses better, by roughly four cases, but that margin does not survive the held-out split: on the 10 TN cases the suppression rules never saw, the point estimate is unchanged and the interval widens threefold to include zero (§7.4).

The advantage of unification at $N=5$ is therefore operational: one pass instead of two, at half the API cost and substantially lower latency, with no loss of F_1 or recall. To that we add one qualitative property a surface-restricted pair cannot produce by construction, the cross-referenced cross-surface finding (§7.4). On accuracy the two architectures are level.

Cost, latency, and two-tier deployment The detector-only row of Table 2 carries the same cost and latency as the full skill, which follows from the implementation rather than from measurement error. SECPRIV executes all five phases in a *single* model call: the phases are structure within one system prompt, not separate round trips. The ablation is an instruction-level override appended to that prompt (“skip Phases 4–5”), so it issues one call with a marginally longer prompt and its cost and latency are indistinguishable from the full skill’s. Disabling the validator buys no compute; it only costs precision. Cost ratios between configurations are meaningful while absolute figures are indicative, and the ratio that matters is that unification removes one of the two passes. Because Haiku is statistically tied with Sonnet on F_1 while trading precision for recall, a two-tier deployment that sends the harder cross-surface cases to Sonnet averages $\sim \$0.016/\text{case}$ if half route up, and $\sim \$0.012$ if a quarter do.

Relation to SAST Deterministic rules remain complementary despite Semgrep’s low coverage on our probe, and stronger baselines should include advisory-specific rules or CodeQL. The skill-as-prompt paradigm has hard limits of its own: no cross-file memory, which sets the probe’s recall ceiling; a schema frozen at prompt-write time [38]; no runtime verification of whether a consent flag is checked or a key is truly KMS-managed [29]; and $\sim 5\text{K}$ tokens of prompt pressure. One failure is stable across every run: an `@require_admin` endpoint is flagged as *auth-bypass* because the skill cannot model the decorator, so auth findings on decorator-protected endpoints warrant a second look.

Limitations The controlled benchmark is small and synthetic (128 self-contained Python files), and the transformation micro-benchmark tests a mechanism rather than sampling a corpus. The cross-domain subset that carries the unification claim contains 8 cases (§7.4), too few for a quantitative claim of cross-surface superiority. Only the real-CVE probe compares detection on equal terms, for the schema reasons set out in §7.1. That probe improves external validity but is security-only, since public CVE-style privacy labels are rare, and its post-patch metric estimates residual noise without guaranteeing the fixed file is defect-free. Sonnet and Haiku also vary model capacity but not provider

family, so these scaffold benefits need re-testing on other families before anyone treats them as model-independent.

9 Conclusion

SECPRIV shows that a single detector–validator skill can cover both the security and privacy halves of source-to-sink review without losing accuracy to a pair of single-surface skills. What unification buys is operational: one pass instead of two, at half the API cost, with F_1 and recall unchanged. The skill itself buys a stable reporting vocabulary and a suppression layer; on real CVEs an unstructured prompt of the same model detects about as well, and precision comes from the validator rather than the detector. Detection stalls near 0.6 on the missing-guard classes our scope excludes, and the cross-surface and privacy claims rest on constructed cases rather than field data. A real privacy corpus is what this line of work needs next. All artifacts are released.

Disclosure of Interests. The authors have no competing interests to declare that are relevant to the content of this article.

References

1. European Parliament and Council. Regulation (EU) 2016/679 (General Data Protection Regulation). *Official Journal of the European Union*, 2016.
2. California State Legislature. California Consumer Privacy Act of 2018 (CCPA). Cal. Civ. Code §§1798.100–1798.199, 2018.
3. MITRE. CWE: Common Weakness Enumeration. <https://cwe.mitre.org>, 2024.
4. B. Chess and G. McGraw. Static analysis for security. *IEEE Security & Privacy*, 2(6):76–79, 2004.
5. Semgrep, Inc. Semgrep: Lightweight static analysis for many languages. <https://semgrep.dev>, 2023.
6. P. Avgustinov, O. de Moor, M. P. Jones, and M. Schäfer. QL: Object-oriented queries on relational data. In *30th European Conference on Object-Oriented Programming (ECOOP)*, Schloss Dagstuhl, 2016.
7. S. Arzt, S. Rasthofer, C. Fritz, E. Bodden, A. Bartel, J. Klein, Y. Le Traon, D. Octeau, and P. McDaniel. FlowDroid: Precise context, flow, field, object-sensitive and lifecycle-aware taint analysis for Android apps. In *PLDI*, 2014.
8. B. Johnson, Y. Song, E. Murphy-Hill, and R. Bowdidge. Why don’t software developers use static analysis tools to find bugs? In *ICSE*, 2013.
9. M. Christakis and C. Bird. What developers want and need from program analysis: An empirical study. In *ASE*, 2016.
10. S. Ouyang, J. M. Zhang, M. Harman, and M. Wang. An empirical study of the non-determinism of ChatGPT in code generation. *ACM Transactions on Software Engineering and Methodology*, 34(2):1–28, 2025.
11. Y. Ding et al. Vulnerability detection with code language models: How far are we? In *ICSE*, 2025 (preprint 2024).
12. E. Beljulji and G. Gür. Large language models in security code review and testing. *Journal of Systems Research*, 5(1), 2025.

13. W. Charoenwet, K. Tantithamthavorn, P. Thongtanunam, H. Y. Lin, M. Jeong, and M. Wu. AgenticSCR: An autonomous agentic secure code review for immature vulnerabilities detection. In *FSE*, 2026.
14. W. Charoenwet, P. Thongtanunam, V.-T. Pham, and C. Treude. Toward effective secure code reviews: An empirical study of security-related coding weaknesses. *Empirical Software Engineering*, 29(4):88, 2024.
15. L. Braz and A. Bacchelli. Software security during modern code review: The developer’s perspective. In *ESEC/FSE*, 2022.
16. L. Braz, C. Aeberhard, G. Çalikli, and A. Bacchelli. Less is more: Supporting developers in vulnerability detection during code review. In *ICSE*, 2022.
17. F. Tang, B. M. Østfold, and M. Bruntink. Identifying personal data processing for code review. *arXiv:2301.01568*, 2023.
18. I. Theophilou and G. M. Kapitsaki. Examining software developers’ needs for privacy enforcing techniques: A survey. In *ACM SAC*, 2026.
19. A. S. George, T. Baskar, and P. B. Srikaanth. Bridging the security skills gap. *Partners Universal Innovative Research Publication*, 3(3), 2025.
20. J. Chen et al. SecureVibeBench: Benchmarking secure vibe coding of AI agents via reconstructing vulnerability-introducing scenarios. *arXiv:2509.22097v4*, 2025.
21. S. Fugkeaw et al. AP2I: Adaptive PII scanning and consent discovery system. In *KST*, 2021.
22. H. Deng, W. Wang, and C. Peng. Ceive: Combating caller ID spoofing on 4G phones. In *ACM MobiCom*, 2018.
23. W. Wang et al. Analyzing the attack landscape of Zigbee-enabled IoT systems and reinstating users’ privacy. In *ACM WiSec*, 2020.
24. F. Cicala, W. Wang, T. Wang, N. Li, E. Bertino, F. Liang, and Y. Yang. Pure: A framework for analyzing proximity-based contact tracing protocols. *ACM Computing Surveys*, 55(1):1–36, 2021.
25. W. Wang et al. Towards efficient privacy-preserving deep packet inspection. In *ESORICS*, 2023.
26. S. Zhao, W. Wang, N. Li, and Z. Lin. Styx: Collaborative and private data processing with TEE-enforced sticky policy. *arXiv preprint arXiv:2604.04082*, 2026.
27. OWASP Foundation. OWASP Top 10:2021 — The ten most critical web application security risks. <https://owasp.org/Top10/>, 2021.
28. MITRE Corporation. 2025 CWE Top 25 most dangerous software weaknesses. <https://cwe.mitre.org/top25/>, 2025.
29. H. Pearce, B. Ahmad, B. Tan, B. Dolan-Gavitt, and R. Karri. Asleep at the keyboard? Assessing the security of GitHub Copilot’s code contributions. In *IEEE S&P*, 2022.
30. Z. Li, D. Zou, S. Xu, X. Ou, H. Jin, S. Wang, Z. Deng, and Y. Zhong. VulDeePecker: A deep learning-based system for vulnerability detection. In *NDSS*, 2018.
31. C. Thapa et al. Transformer-based language models for software vulnerability detection. In *ACSAC*, 2022.
32. L. Sweeney. *k*-Anonymity: A model for protecting privacy. *International Journal of Uncertainty, Fuzziness and Knowledge-Based Systems*, 10(5):557–570, 2002.
33. A. Cavoukian. Privacy by design: The 7 foundational principles. *Information and Privacy Commissioner of Ontario*, 2009.
34. D. J. Solove. A taxonomy of privacy. *University of Pennsylvania Law Review*, 154(3):477–564, 2006.
35. M. Chen et al. Evaluating large language models trained on code. *arXiv:2107.03374*, 2021.
36. S. Zhao, D. Wang, K. Zhang, J. Luo, Z. Li, and L. Li. Is vibe coding safe? Benchmarking vulnerability of agent-generated code in real-world tasks. *arXiv:2512.03262*, 2025.
37. J. Yang, C. E. Jimenez, A. Wettig, K. Lieret, S. Yao, K. Narasimhan, and O. Press. SWE-agent: Agent-computer interfaces enable automated software engineering. In *NeurIPS*, 2024.

38. Z. Ji, N. Lee, R. Frieske, T. Yu, D. Su, Y. Xu, E. Ishii, Y. J. Bang, A. Madotto, and P. Fung. Survey of hallucination in natural language generation. *ACM Computing Surveys*, 55(12):1–38, 2023.
39. X. Zhou, S. Cao, X. Sun, and D. Lo. Large language model for vulnerability detection and repair: Literature review and the road ahead. *ACM Transactions on Software Engineering and Methodology*, 34(5):1–31, 2025.
40. P. Ferrara et al. Static analysis for discovering IoT vulnerabilities. *International Journal on Software Tools for Technology Transfer*, 2021.
41. GitHub. CodeQL: Semantic code analysis engine. <https://codeql.github.com/>, 2024.
42. Privado, Inc. Privado: Code scanning for data privacy. <https://www.privado.ai>, 2024.